

Checklist de Desarrollo — Sistema Empresarial de Tickets & Chat

Convenciones de estado

Símbolo	Significado
[]	Pendiente
[x]	Completado
[-]	En progreso
[~]	Bloqueado / depende de otro

FASE 0 — Preparación del entorno

0.1 Repositorio y estructura base

- [] Crear repositorio Git (GitHub / GitLab privado)
- [] Crear estructura de monorepo: `apps/api`, `apps/web`, `apps/mobile_flutter`, `packages/`, `infrastructure/`
- [] Configurar `.gitignore` para Node, Flutter, env files
- [] Crear `.env.example` con todas las variables documentadas
- [] Configurar rama `main` protegida + rama `develop` para trabajo activo
- [] Crear `README.md` con instrucciones de arranque local

0.2 Infraestructura local Docker

- [] Crear `docker-compose.dev.yml` con: `postgres`, `redis`, `minio`, `n8n`
- [] Crear `docker-compose.prod.yml` completo
- [] Configurar `infrastructure/nginx/nginx.conf` con reverse proxy y SSL
- [] Configurar `infrastructure/postgres/init.sql` con `schema.sql`
- [] Crear bucket `helpdesk` en MinIO al iniciar
- [] Verificar que todos los servicios arrancan con `docker compose up -d`
- [] Probar conexión a PostgreSQL desde fuera del contenedor
- [] Probar acceso a consola MinIO en `:9001`

0.3 Configuración EasyPanel (producción)

- ☐ Crear aplicación `api` en EasyPanel apuntando a imagen Docker del backend
- ☐ Crear aplicación `web` en EasyPanel
- ☐ Configurar base de datos PostgreSQL en EasyPanel (o externa)
- ☐ Configurar Redis en EasyPanel
- ☐ Configurar MinIO como servicio
- ☐ Configurar n8n como servicio
- ☐ Configurar variables de entorno en EasyPanel para cada servicio
- ☐ Habilitar SSL con Let's Encrypt en EasyPanel
- ☐ Configurar dominio o IP pública de la empresa
- ☐ Verificar reglas de firewall: 443 solo desde IP empresa, 22 solo desde IP admin

FASE 1 — Backend NestJS (MVP)

1.1 Proyecto base NestJS

- ☐ Crear proyecto con `nest new api`
- ☐ Instalar dependencias: `@nestjs/config`, `@nestjs/jwt`, `@nestjs/passport`, `@nestjs/websockets`, `@nestjs/platform-socket.io`
- ☐ Instalar Prisma: `prisma`, `@prisma/client`
- ☐ Instalar validación: `class-validator`, `class-transformer`
- ☐ Instalar BullMQ: `@nestjs/bullmq`, `bullmq`
- ☐ Instalar MinIO SDK: `minio`
- ☐ Instalar utilidades: `bcrypt`, `nanoid`, `dayjs`, `lodash`
- ☐ Configurar `ValidationPipe` global
- ☐ Configurar `AllExceptionsFilter` global
- ☐ Configurar `TransformInterceptor` global
- ☐ Configurar Swagger con `@nestjs/swagger`
- ☐ Configurar CORS para web y app móvil
- ☐ Configurar prefijo global `/api/v1`

1.2 Prisma schema y migración inicial

- ☐ Escribir `schema.prisma` completo basado en el modelo de BD
- ☐ Ejecutar `prisma migrate dev --name init`
- ☐ Ejecutar `prisma generate`

- [] Crear seeder para: roles, permisos, departamentos base, estados, prioridades, storage_providers
- [] Crear script `npm run seed` funcional
- [] Verificar integridad de datos en BD tras seed

1.3 Módulo config y env

- [] Crear `config/app.config.ts`
- [] Crear `config/database.config.ts`
- [] Crear `config/redis.config.ts`
- [] Crear `config/jwt.config.ts`
- [] Crear `config/storage.config.ts`
- [] Validar variables de entorno con Joio Zod al arrancar

1.4 Módulo auth

- [] POST `/auth/request-otp` — recibe cédula, genera OTP de 6 dígitos, guarda hash, envía por WhatsApp
- [] POST `/auth/verify-otp` — valida OTP, retorna access token + refresh token
- [] POST `/auth/refresh` — valida refresh token, emite nuevo par de tokens
- [] POST `/auth/logout` — revoca refresh token activo
- [] Implementar `JwtStrategy` con Passport
- [] Implementar `JwtAuthGuard`
- [] Implementar `RolesGuard` con decorador `@Roles(...)`
- [] Implementar control de intentos OTP (máx 3) y expiración (5 minutos)
- [] Prueba manual: solicitar OTP y verificar con token inválido, expirado y correcto

1.5 Módulo users

- [] GET `/me` — perfil del usuario autenticado
- [] GET `/users` — lista paginada (solo admin/supervisor)
- [] GET `/users/:id` — detalle
- [] POST `/users` — crear empleado (admin)
- [] PATCH `/users/:id` — editar (admin)
- [] PATCH `/users/:id/status` — activar / desactivar
- [] Relacionar usuario con departamentos al crear
- [] Relacionar usuario con roles al crear

1.6 Módulo departments

- [] GET /departments — listar departamentos activos
- [] GET /departments/:id — detalle con usuarios y plantillas
- [] POST /departments — crear (admin)
- [] PATCH /departments/:id — editar
- [] GET /departments/:id/templates — plantillas asignadas al departamento
- [] POST /departments/:id/templates — asociar plantilla a departamento

1.7 Módulo templates

- [] GET /templates — listar con filtros por tipo y estado
- [] GET /templates/:id — detalle con campos
- [] POST /templates — crear plantilla con campos dinámicos
- [] PATCH /templates/:id — editar (crea nueva versión si ya tiene tickets asociados)
- [] POST /templates/:id/fields — agregar campo
- [] PATCH /templates/:id/fields/:fieldId — editar campo
- [] DELETE /templates/:id/fields/:fieldId — eliminar campo
- [] Implementar validador de value_json contra esquema de template_fields

1.8 Módulo inventory

- [] GET /assets — listar con filtros por departamento, tipo, estado, búsqueda por serial/nombre
- [] GET /assets/:id — detalle del activo
- [] GET /assets/:id/lifecycle — historial completo del activo
- [] POST /assets — registrar activo
- [] PATCH /assets/:id — editar activo
- [] POST /assets/:id/lifecycle — agregar entrada manual a hoja de vida
- [] GET /assets/qr/:code — buscar activo por QR
- [] Generación de código QR como imagen PNG

1.9 Módulo tickets

- [] GET /tickets — listar con filtros por estado, prioridad, área, asignado, fecha
- [] GET /tickets/:id — detalle completo
- [] GET /tickets/:id/timeline — eventos del ticket ordenados cronológicamente
- [] POST /tickets — crear ticket (valida plantilla y campos requeridos)
- [] PATCH /tickets/:id — editar datos del ticket

- [] POST /tickets/:id/assign — asignar a técnico (supervisor/admin)
- [] POST /tickets/:id/change-status — cambiar estado (válida transición permitida)
- [] POST /tickets/:id/close — cerrar ticket (exige resolución)
- [] POST /tickets/:id/comments — agregar comentario interno o público
- [] POST /tickets/:id/attachments — adjuntar archivo
- [] Al cerrar ticket con activo vinculado: crear `asset_lifecycle_entry` automáticamente
- [] Calcular `sla_due_at` al crear el ticket según política SLA del departamento

1.10 Módulo attachments (almacenamiento)

- [] Implementar interfaz `IStorageProvider` con métodos: `upload`, `getUrl`, `delete`
- [] Implementar `MinioStorageService` usando `storage_providers` de BD
- [] Implementar `S3StorageService` como alternativa
- [] POST /attachments/presign — URL pre-firmada para subida directa desde cliente
- [] GET /attachments/:id/url — URL de descarga con expiración
- [] Leer proveedor activo (`is_default = true`) desde BD, no desde variables de entorno

1.11 Módulo chat (WebSocket)

- [] GET /chat/channels — listar canales del usuario autenticado
- [] GET /chat/channels/:id/messages — historial paginado (cursor-based)
- [] POST /chat/channels — crear canal
- [] POST /chat/channels/:id/members — agregar miembro
- [] Implementar ChatGateway con Socket.IO
- [] Evento `chat:message` — enviar mensaje a canal
- [] Evento `chat:typing` — indicador de escritura
- [] Evento `chat:read` — marcar mensajes como leídos
- [] Unir usuario a sus canales al conectar WebSocket
- [] Autenticar WebSocket por JWT en header o query param
- [] Crear canal tipo `ticket` automáticamente al crear ticket
- [] Crear canales de departamento al crear un departamento

1.12 Módulo notifications

- [] Implementar `WhatsAppService` — enviar mensajes por API WhatsApp Business
- [] Implementar `EmailService` — enviar por SMTP / Nodemailer
- [] Implementar `PushService` — Firebase Cloud Messaging (FCM)
- [] Implementar `InAppService` — notificación interna por WebSocket

- [] `NotificationsService` orquesta el canal correcto según preferencia/disponibilidad
- [] Disparar notificación al crear ticket (solicitante + supervisor del área)
- [] Disparar notificación al asignar ticket (técnico asignado)
- [] Disparar notificación al cambiar estado (solicitante)
- [] Disparar notificación al cerrar ticket (solicitante)
- [] Disparar alerta SLA vencido (supervisor)

1.13 Módulo webhooks

- [] `POST /webhooks/tickets` — endpoint inbound para recibir eventos de n8n o sistemas externos
- [] Validación de firma HMAC o Bearer Token en entrada
- [] `WebhookDeliveryProcessor` (BullMQ) — reintentos exponenciales para webhooks salientes
- [] Disparar webhook al crear ticket
- [] Disparar webhook al cerrar ticket
- [] Disparar webhook al escalar ticket (SLA vencido)
- [] Registrar cada intento en `webhook_deliveries`

1.14 Módulo SLA

- [] Calcular `sla_due_at` al crear ticket según `sla_policies` del departamento + prioridad
- [] `Job BullMQ SlaMonitorProcessor` — revisa cada 5 min tickets abiertos próximos a vencer
- [] Disparar alerta cuando queda 20% del tiempo SLA
- [] Marcar ticket como `sla_breached` cuando se supera el tiempo
- [] Escalar ticket automáticamente al supervisor si no tiene técnico asignado

1.15 Módulo audit

- [] `AuditInterceptor` — intercepta todas las mutaciones y escribe en `audit_logs`
- [] `GET /audit` — consultar logs (solo admin/auditor) con filtros
- [] Registrar: usuario, entidad, acción, IP, valores antes/después

1.16 Módulo reports

- [] `GET /reports/tickets/summary` — tickets por estado, área, prioridad en rango de fechas
- [] `GET /reports/tickets/by-technician` — productividad por técnico
- [] `GET /reports/assets/maintenance` — historial de mantenimiento por activo/área
- [] `GET /reports/sla` — cumplimiento de SLA por área y prioridad

- [] Exportación a CSV
- [] Exportación a PDF (usando `pdfmake` o `puppeteer`)

1.17 Módulo dashboard

- [] GET `/dashboard/summary` — KPIs generales para admin
- [] GET `/dashboard/department/:id` — KPIs por departamento para supervisor
- [] GET `/dashboard/technician` — carga de trabajo del técnico autenticado
- [] GET `/dashboard/requester` — mis tickets para solicitante

1.18 Módulo mobile (endpoints optimizados Flutter)

- [] GET `/mobile/tickets` — lista compacta optimizada para móvil
- [] POST `/mobile/sync` — sincronización offline (lote de acciones pendientes)
- [] GET `/mobile/assets/scan/:code` — búsqueda rápida por QR o serial
- [] POST `/mobile/tickets/:id/photos` — subida múltiple de fotos desde app

1.19 API Tokens (integraciones externas)

- [] POST `/api-tokens` — crear token con scopes (admin)
- [] GET `/api-tokens` — listar tokens activos
- [] DELETE `/api-tokens/:id` — revocar token
- [] `ApiTokenGuard` — autenticación alternativa por header `X-API-Token`

FASE 1 — Frontend Web (MVP)

2.1 Proyecto base React + Vite

- [] Crear proyecto con Vite + React + TypeScript
- [] Instalar TailwindCSS + configuración
- [] Instalar React Router v6
- [] Instalar Zustand para estado global
- [] Instalar Axios + interceptors JWT (auto-refresh)
- [] Instalar Socket.IO client
- [] Configurar PWA con `vite-plugin-pwa`
- [] Configurar proxy de API en desarrollo

2.2 Módulo auth web

- ☐ Pantalla de login: ingreso de cédula
- ☐ Pantalla de verificación OTP con temporizador
- ☐ Manejo de sesión: guardar token en memoria, refresh automático
- ☐ Ruta protegida por rol
- ☐ Logout con limpieza de sesión

2.3 Layout general

- ☐ Sidebar de navegación adaptado por rol
- ☐ Topbar con búsqueda global, notificaciones y perfil
- ☐ Responsive: sidebar colapsable en mobile
- ☐ Modo oscuro / claro

2.4 Dashboard principal

- ☐ KPIs por rol (tickets abiertos, SLA, tiempo promedio de resolución)
- ☐ Gráfica de tickets por estado
- ☐ Gráfica de carga por técnico
- ☐ Panel de tickets recientes

2.5 Módulo tickets web

- ☐ Lista de tickets con filtros, ordenamiento y paginación
- ☐ Crear ticket: selecciona departamento → carga plantilla → campos dinámicos → activo opcional
- ☐ Detalle de ticket: timeline, comentarios, adjuntos, cambio de estado
- ☐ Asignación a técnico (supervisor/admin)
- ☐ Cierre con resolución y checklist
- ☐ Vista por rol: diferente para solicitante, técnico, supervisor, admin

2.6 Módulo inventario web

- ☐ Lista de activos por departamento con filtros
- ☐ Detalle de activo con hoja de vida completa
- ☐ Crear / editar activo
- ☐ Vista del historial de mantenimiento
- ☐ Generación e impresión de código QR del activo

2.7 Módulo chat web

- ☐ Lista de canales del usuario
- ☐ Vista de conversación con mensajes en tiempo real (WebSocket)
- ☐ Envío de texto y archivos adjuntos
- ☐ Indicador de escritura y mensajes leídos
- ☐ Canal de ticket embebido en el detalle del ticket

2.8 Módulo departamentos y plantillas (admin)

- ☐ CRUD de departamentos
- ☐ CRUD de plantillas con editor de campos dinámicos
- ☐ Asociar plantillas a departamentos
- ☐ Vista previa del formulario generado por la plantilla

2.9 Módulo admin

- ☐ Gestión de usuarios y roles
- ☐ Gestión de tokens de API
- ☐ Configuración de webhooks
- ☐ Configuración de storage providers (MinIO)
- ☐ Configuración de SLA por área/prioridad
- ☐ Logs de auditoría

FASE 1 — App Flutter (MVP)

3.1 Proyecto base Flutter

- ☐ Crear proyecto flutter `create mobile_flutter`
- ☐ Instalar dependencias: `dio`, `flutter_riverpod` o `bloc`, `hive_flutter` (offline), `socket_io_client`, `firebase_messaging`, `mobile_scanner` (QR), `image_picker`, `go_router`
- ☐ Configurar flavors o entornos: `dev`, `prod`
- ☐ Configurar `dio` con interceptors para JWT y refresh automático
- ☐ Configurar FCM para push notifications
- ☐ Configurar tema y estilos globales

3.2 Auth Flutter

- ☐ Pantalla ingreso de cédula
- ☐ Pantalla verificación OTP con teclado numérico grande
- ☐ Guardar tokens en `flutter_secure_storage`
- ☐ Refresh automático de token
- ☐ Logout y limpieza de sesión local

3.3 Navegación por rol

- ☐ Detectar rol al iniciar sesión
- ☐ Redirigir a home apropiado según rol
- ☐ Bottom navigation bar con ítems por rol
- ☐ Rutas protegidas

3.4 Funcionalidades solicitante (Flutter)

- ☐ Pantalla principal: mis tickets con estado visual
- ☐ Crear ticket: seleccionar área, completar formulario, adjuntar foto
- ☐ Detalle de ticket: barra de progreso por estado, chat integrado
- ☐ Push notification al cambiar estado del ticket

3.5 Funcionalidades técnico (Flutter)

- ☐ Bandeja de tickets asignados con prioridad visual
- ☐ Detalle del ticket con checklist de intervención
- ☐ Cambiar estado desde el campo
- ☐ Captura y subida de fotos (antes/durante/después)
- ☐ Escáner QR para identificar activo
- ☐ Consulta de historial del activo escaneado
- ☐ Cerrar ticket con resolución escrita desde el teléfono
- ☐ Modo offline básico: cambios en cola local, sync al reconectar

3.6 Funcionalidades supervisor (Flutter)

- ☐ Dashboard del área con KPIs
- ☐ Lista de tickets sin asignar con alerta
- ☐ Aprobar, asignar o reasignar ticket con un toque
- ☐ Chat del equipo
- ☐ Push notifications para tickets sin técnico > X minutos

3.7 Funcionalidades admin (Flutter)

- ☐ Alertas críticas del sistema
- ☐ Aprobación rápida de tickets escalados
- ☐ Métricas generales de uso

3.8 Funcionalidades auditor (Flutter)

- ☐ Reportes en vista de solo lectura
- ☐ Exportación de datos a PDF

3.9 Offline sync

- ☐ Cola local con Hive para acciones offline (cerrar ticket, cambiar estado)
- ☐ Sincronización automática al recuperar conectividad
- ☐ Indicador visual de modo offline activo

FASE 2 — Funcionalidad avanzada

4.1 Automatizaciones

- ☐ Configurar n8n y conectar con endpoint `/webhooks/tickets`
- ☐ Workflow n8n: ticket creado → enviar WhatsApp al solicitante
- ☐ Workflow n8n: ticket sin asignar > 30 min → alerta al supervisor
- ☐ Workflow n8n: SLA vencido → notificación a admin
- ☐ Workflow n8n: ticket cerrado → registro en sistema externo (opcional)

4.2 Clasificación por IA

- ☐ Endpoint POST `/ai/suggest` — recibe descripción del ticket, retorna área, prioridad sugerida
- ☐ Integrar en el formulario de creación como sugerencia (no obligatoria)
- ☐ Usar OpenAI API o Ollama local según disponibilidad

4.3 Búsqueda avanzada

- ☐ Instalar OpenSearch en Docker Compose
- ☐ Indexar tickets y mensajes de chat
- ☐ GET `/search?q=` — búsqueda full-text en tickets, activos y chat

4.4 Métricas avanzadas y Observabilidad

- ☐ Configurar Prometheus + Grafana en Docker Compose
- ☐ Exponer métricas del backend en `/metrics`
- ☐ Dashboard Grafana: tickets por hora, SLA en vivo, WebSocket connections
- ☐ Configurar Loki para logs centralizados
- ☐ Alertas en Grafana por SLA vencido o errores de webhook

FASE 3 — Plataforma expandible

5.1 Nuevos departamentos dinámicos

- ☐ UI para crear nuevo departamento + asignar plantillas + configurar flujo desde panel admin
- ☐ Sin necesidad de despliegue de código para nuevas áreas

5.2 Distribución app Flutter

- ☐ Generar APK/AAB para distribución interna
- ☐ Configurar descarga desde servidor interno: `https://app.empresa.com/install`
- ☐ Opcional: publicar en Google Play interno (track cerrado)

5.3 Auditoría avanzada

- ☐ Reportes de auditoría exportables para contraloría
- ☐ Trazabilidad completa por activo: quién tocó, cuándo, qué resolvió

5.4 Multi-empresa (futuro)

- ☐ Agregar campo `tenant_id` si el sistema se quiere volver SaaS interno/externo
- ☐ Row-level security en PostgreSQL por tenant

Seguridad — Checklist transversal

- ☐ HTTPS obligatorio en todos los entornos (Let's Encrypt)
- ☐ JWT con expiración corta (15 min access, 7 días refresh)
- ☐ Rate limiting por IP en endpoints de OTP (máx 3 solicitudes/hora)
- ☐ Firewall: 443 solo IP empresa + IPs autorizadas, 22 solo IP admin
- ☐ Sanitización de todos los inputs (class-validator + SQL parameterizado vía Prisma)
- ☐ CORS configurado con lista blanca de orígenes
- ☐ Adjuntos: validar MIME type real (no solo extensión), límite de tamaño

- [] MinIO: buckets privados, acceso solo por URLs pre-firmadas con expiración
- [] Tokens de API: hash en BD (bcrypt), nunca almacenar en texto plano
- [] Logs de auditoría inmutables (sin DELETE en tabla `audit_logs`)
- [] Backups automáticos diarios de PostgreSQL y MinIO
- [] Escaneo periódico de dependencias: `npm audit`, `flutter pub outdated`
- [] Variables de entorno nunca en código fuente ni repositorio Git

Testing — Checklist

- [] Tests unitarios de servicios críticos: `auth.service`, `tickets.service`, `lifecycle.service`
- [] Tests E2E de flujo principal: crear ticket → asignar → cerrar → verificar hoja de vida
- [] Test de carga básico: 1000 conexiones WebSocket simultáneas con `artillery` o `k6`
- [] Test de regresión: plantillas dinámicas con diferentes tipos de campos
- [] Prueba de red lenta en Flutter (modo avión → reconectar → verificar sync)
- [] Prueba de escaneo QR en Flutter con activos reales del inventario